

Adapting to Any Screen

Before you write a single line of responsive CSS, you must tell the browser to stop lying about its width.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

THE PROBLEM

By default, mobile browsers pretend they are a desktop screen (usually 980px wide) and shrink the page down to fit, making text unreadable.

THE SOLUTION

This meta tag forces the browser to report its actual physical width, allowing your CSS media queries to work correctly.

Media Queries

Media queries allow you to apply CSS rules only when certain conditions are met, such as the screen width exceeding a specific breakpoint.

THE MECHANISM

A media query acts like an `if` statement in CSS. If the browser matches the condition (e.g., it is wider than 768px), the styles inside the block are applied.

```
@media (min-width: 768px) { /* Styles applied on tablets & up */ }
```

BREAKPOINTS

A "breakpoint" is the specific width where your layout needs to change because the current design no longer looks good or functions well.

```
.layout { display: flex; flex-direction: column; }  
@media (min-width: 768px) { .layout { display: grid; grid-template-columns: 1fr 1fr; } }
```

MOBILE (< 768PX)

1

2

TABLET/DESKTOP (≥ 768PX)

1

2

Mobile-First Workflow

Write your default styles for small screens first, then use `min-width` media queries to add complexity as the screen gets wider.

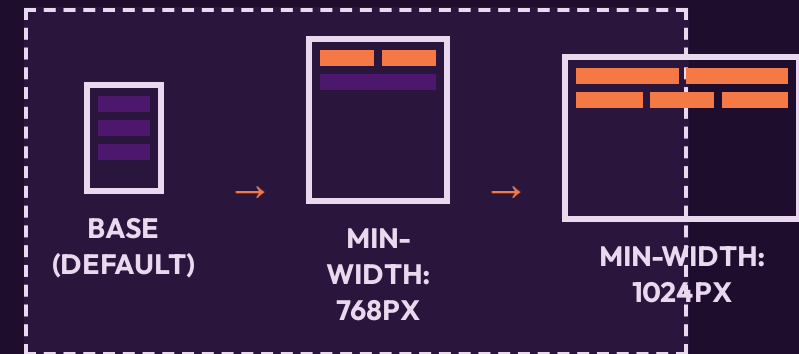
WHY MOBILE-FIRST?

Mobile layouts are usually simpler (often a single column). It is much easier to start simple and add complexity (like grids and sidebars) than to start complex and try to strip things away.

THE GOLDEN RULE

Your base CSS (outside of any media query) should be your mobile view. You should almost exclusively use `min-width` queries, moving upwards.

```
/* 1. Base */ .nav { display: none; } /* hide nav */
.content { padding: 1rem; }
/* 2. Tablet & Up */ @media (min-width: 768px) {
  .nav{display:flex;} }
/* 3. Desktop & Up */ @media (min-width: 1024px) {
  .content{display:grid;} }
```



Fluid Typography

Instead of jumping between fixed font sizes at breakpoints, use `clamp()` to make text scale smoothly with the screen width.

VIEWPORT UNITS

Units like `vw` (viewport width) scale perfectly with the screen. `1vw` is exactly 1% of the screen's width. However, using pure `vw` for text means it gets too small on mobile and too huge on desktop.

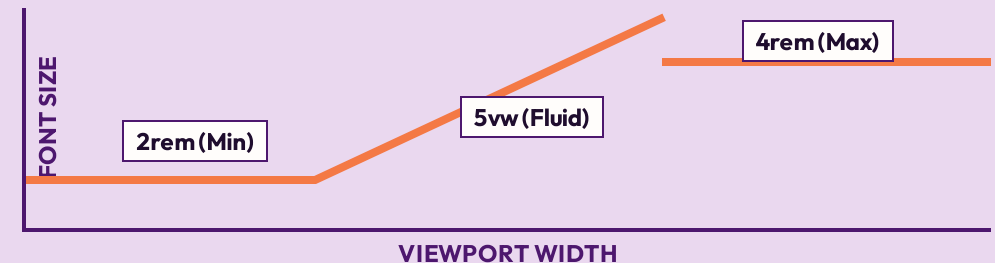
THE CLAMP FUNCTION

`clamp()` takes three values: a minimum size, a preferred size (usually a fluid viewport unit), and a maximum size. It locks the value within those boundaries.

```
clamp(MIN, PREFERRED, MAX);
```

```
h1 {  
  font-size: clamp(2rem, 5vw, 4rem);  
}
```

HOW CLAMP WORKS



Responsive Images

Images are inherently rigid. We must use CSS to ensure they scale down to fit their containers without stretching, squishing, or overflowing.

THE GOLDEN RULE

Setting `max-width: 100%` ensures an image will never be wider than its parent. Use `height: auto` to preserve aspect ratio.

OBJECT-FIT

When an image must fill a box (avatar, banner), `object-fit: cover` crops without distortion — like `background-size: cover` for `<img>`.

```
/* Global reset */  
img {  
  max-width: 100%;  
  height: auto;  
  display: block;  
}
```

OBJECT-FIT: COVER



OBJECT-FIT: CONTAIN



Flex Wrapping

Allow items to naturally flow to the next line when they run out of space, creating responsive layouts without needing media queries.

THE DEFAULT BEHAVIOR

By default, Flexbox tries to fit all items onto a single line (`flex-wrap: nowrap`).

WRAPPING AND BASIS

With `flex-wrap: wrap` and a `flex-basis`, items will wrap when they can no longer keep their basis.

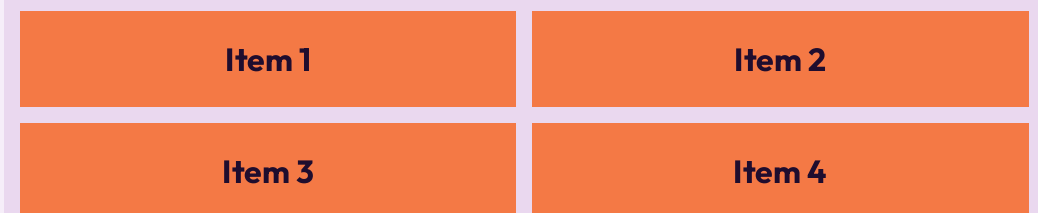
```
.container { display: flex; flex-wrap: wrap; gap: 16px; }  
.item { /* flex: 1 1 200px */ flex: 1 1 200px; }
```

FLEX-WRAP: NOWRAP (DEFAULT)



Items shrink to stay on one line.

FLEX-WRAP: WRAP



Items wrap when they reach their basis.

Grid Auto-Fit and Minmax

Responsive grids that adjust the column count based on available space.

THE MAGIC FORMULA

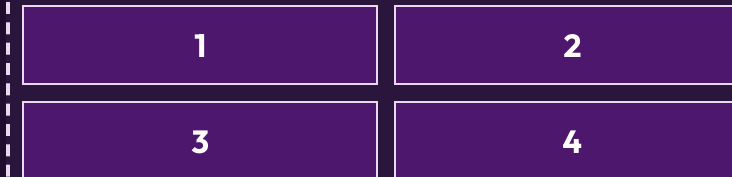
Use `repeat(auto-fit, minmax())` so the browser fits as many columns as the minimum allows.

```
.card-grid {  
  display: grid;  
  gap: 16px;  
  grid-template-columns: repeat(auto-fit,  
minmax(250px, 1fr));  
}
```

WIDE SCREEN (FITS 3 COLUMNS OF AT LEAST 250PX)



MEDIUM SCREEN (FITS 2 COLUMNS)



SMALL SCREEN (FITS 1 COLUMN)



Container Queries

Style components based on the size of their parent, not the viewport.

THE COMPONENT PROBLEM

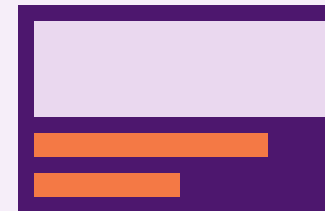
Media queries only know about the screen. A card in a narrow sidebar may get the wrong (wide) layout.

THE CONTAINER SOLUTION

Mark a parent as a container, then use `@container` to adapt its children when that container grows.

```
/* Define a container */  
.card-wrapper { container-type: inline-size; }  
  
/* Query the container */  
.card { display: flex; flex-direction: column; }  
@container (min-width: 400px) { .card { flex-direction: row; } }
```

SIDEBAR (< 400PX)



MAIN AREA (≥ 400PX)



Logical Properties

Stop thinking in physical directions (top, right, bottom, left) and start thinking in flow directions (block and inline).

BLOCK VS. INLINE

The **block** axis is the direction blocks stack (top-to-bottom in English).
The **inline** axis is the direction text flows (left-to-right in English).

WHY IT MATTERS

If a user translates your site to a right-to-left language (like Arabic), physical properties (`margin-left`) break the layout. Logical properties (`margin-inline-start`) automatically flip to match the new reading direction.

SHORTHANDS

Logical properties give us powerful new shorthands. `padding-inline` sets both left and right padding at once. `margin-block` sets both top and bottom.

```
/* The Old Physical Way */  
.card { margin-top: 16px; margin-bottom: 16px;  
padding-left: 24px; padding-right: 24px; }  
  
/* The Modern Logical Way */  
.card { margin-block: 16px; padding-inline: 24px; }
```



Testing Responsive Designs

A responsive design is only as good as it looks on actual screens. You must test continuously throughout the development process.

BROWSER DEVTOOLS

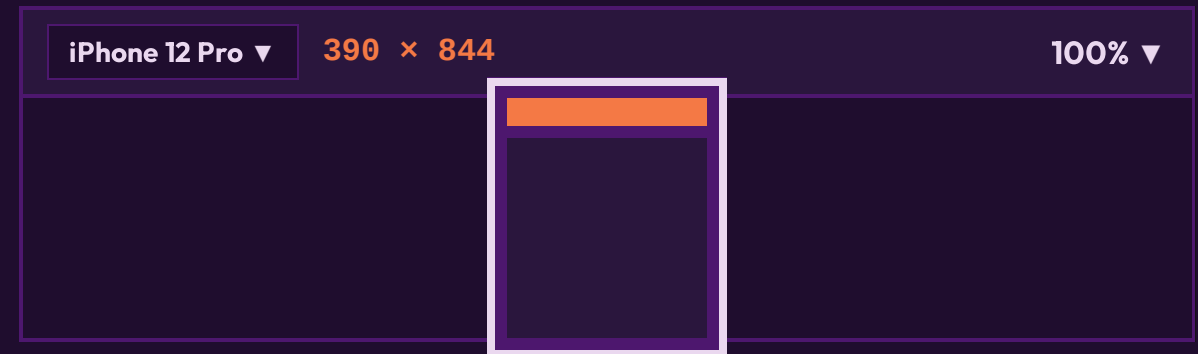
Every modern browser includes a "Device Mode" in its developer tools. This allows you to simulate specific devices (like an iPhone or Pixel) and test touch interactions without leaving your desktop.

THE "SQUISH" TEST

Don't just test at specific breakpoints. Slowly resize your browser window from wide to narrow. Look for the exact pixel widths where the design breaks, and add breakpoints there.

REAL DEVICES

Simulators are not perfect. They cannot replicate true rendering engines, actual touch targets, or real-world performance. Always test on at least one real physical mobile device.



RESPONSIVE CHECKLIST

- ☐ No horizontal scrolling (content fits width).
- ☐ Text is readable without zooming (min 16px).
- ☐ Tap targets (buttons/links) are large enough.
- ☐ Images scale down without distortion.

Common Responsive Patterns

Most responsive layouts rely on a few standard patterns to adapt content across different screen sizes.

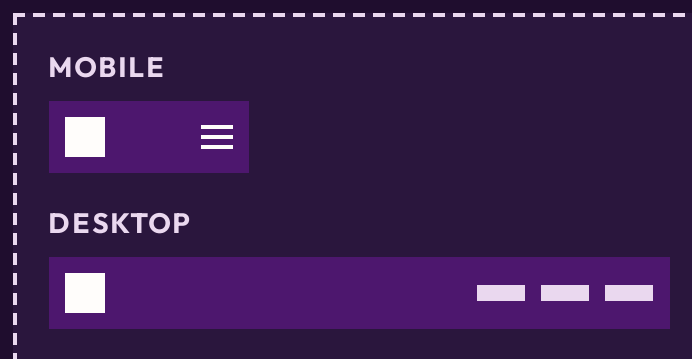
STACKING COLUMNS

Content stacks vertically on narrow screens and flows into multiple columns on wider screens.



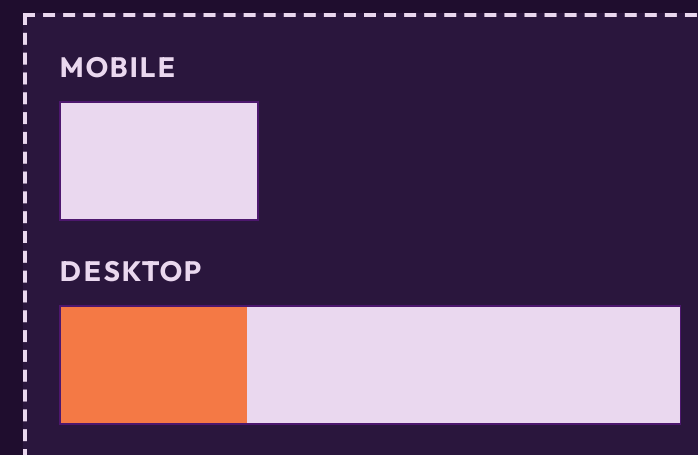
NAVIGATION

Links are hidden behind a toggle button on mobile, and displayed horizontally on desktop.



OFF-CANVAS

Secondary content (like a sidebar) slides in on mobile, but remains permanently visible on desktop.



Lab: Responsive Portfolio

Apply fluid typography, flexible grids, and media queries to build a personal portfolio that looks professional on every device.

THE BRIEF

You have the HTML structure and design tokens. Write responsive CSS so the portfolio adapts from small phones to large desktops.

KEY FOCUS AREAS

• Mobile-first media queries • Grid auto-fit for galleries • Flexbox for nav • Fluid typography for headings

DELIVERABLES

- A responsive navigation that switches from a stacked menu to a horizontal row.
- A project gallery that automatically adjusts its column count based on screen width.
- Responsive images that scale without distortion.

SUCCESS CRITERIA

- Zero horizontal scrolling at any viewport width.
- Text remains readable (minimum 16px) on small screens.